

Industrial Tool

Git & Github

Introduction of Git :-

- Git is a distributed version control system (VCS).
- It is used to track changes in source code.
- Git helps during software development.
- It allows multiple developers to collaborate.
- It manages different versions of code.
- It enables rollback to previous versions when needed.



Open source and free



Public and private repository hosting



Can work as a standalone program, a client, or a server



Distributed architecture



Fast and efficient

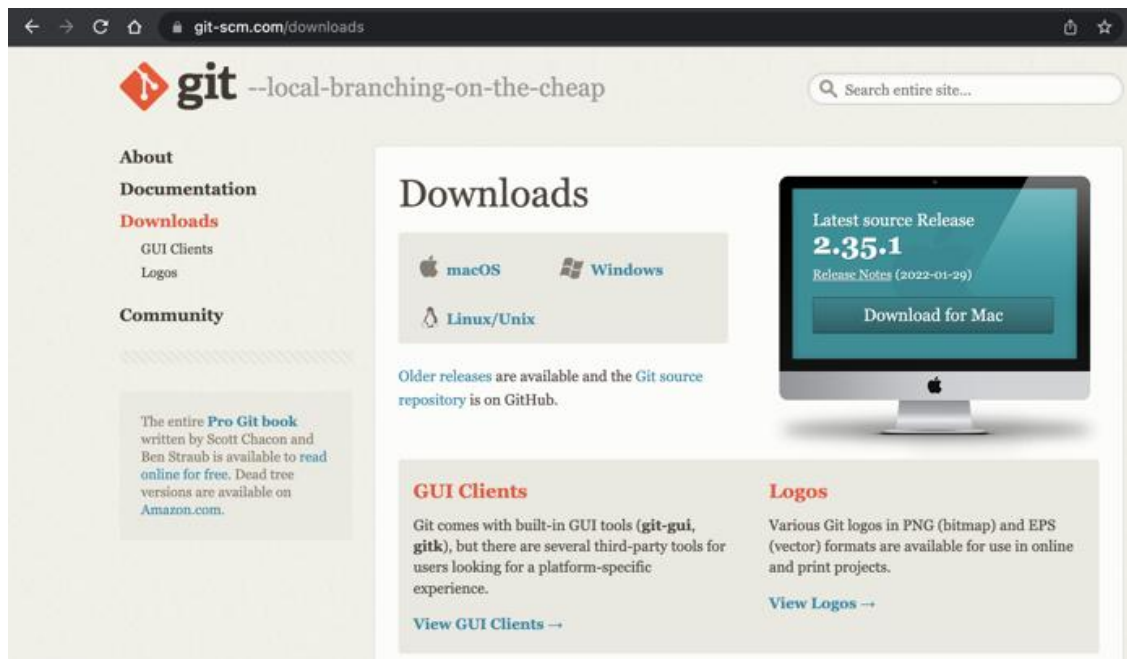


Does not depend on a centralized server

1. **Collaboration:** Multiple developers can work together and merge changes easily.
2. **History Tracking:** Revert to previous versions whenever needed.
3. **Branching & Merging:** Develop features separately and merge them safely.
4. **Distributed Development:** Each developer has a full copy of the repo.
5. **Incremental Changes:** Only differences are stored, saving space and simplifying updates.

Steps for Git Environment Setup :-

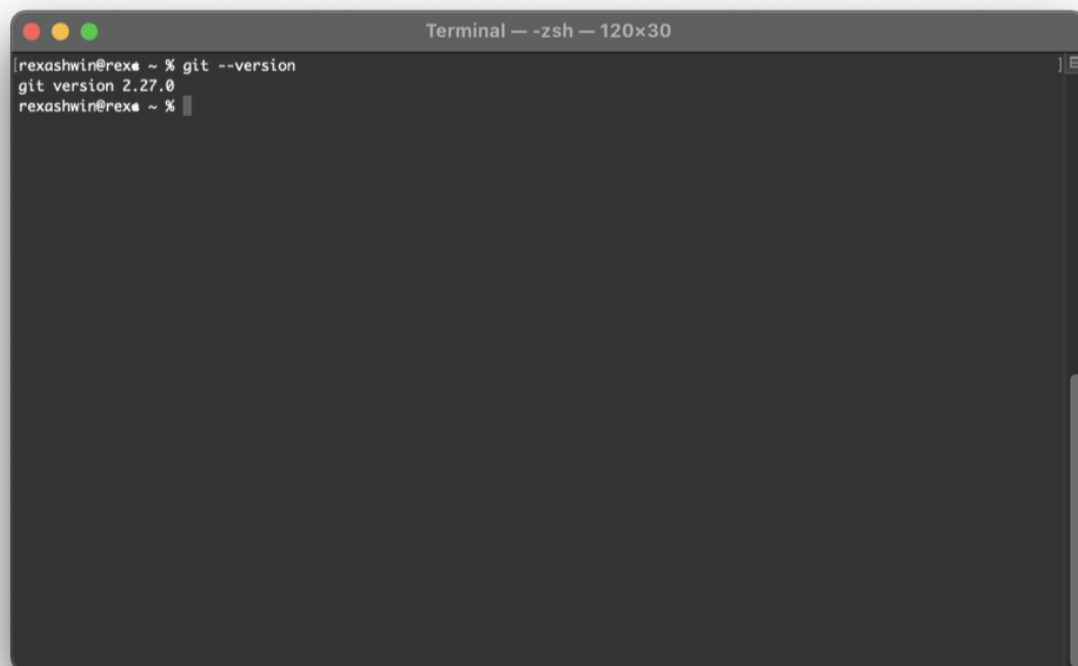
Step 1. Go to git-scm.com/downloads and download as per your system.



git-scm/download webpage

Step 2. If you are using Debian-based GNU/Linux distribution, all you have to do is command ***sudo apt-get install git-core***.

Step 3. Whatever command line you are using, enter "git --version" if it gives back the Git version you are ready to go.



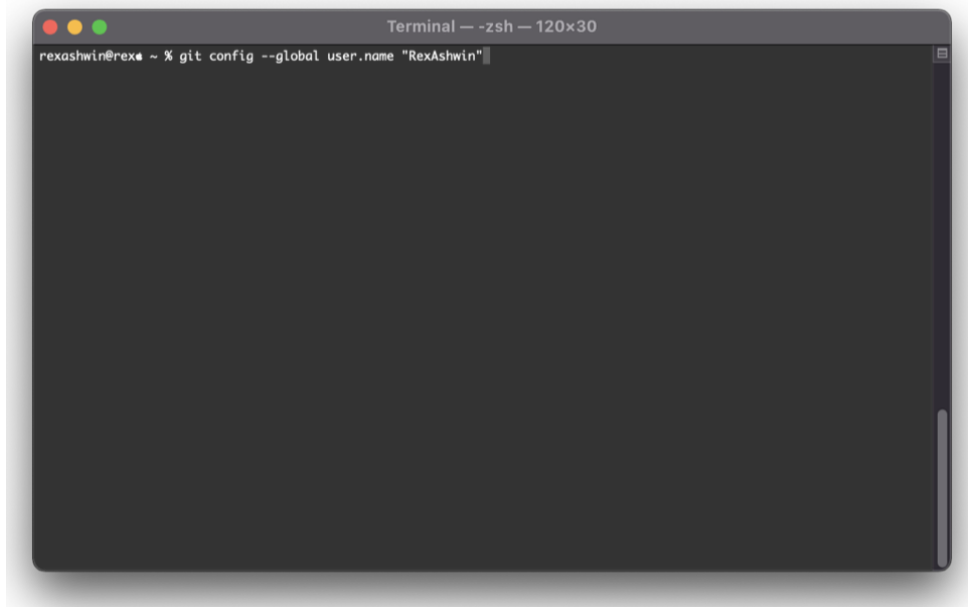
git version check

Setting Git Environment :-

Git uses **git config** to set configuration values. The **--global** option applies settings to all repositories, while without it, settings apply only to the current repository.

Setting username

This is for customizing our username, which will tell who is working on the repositories.

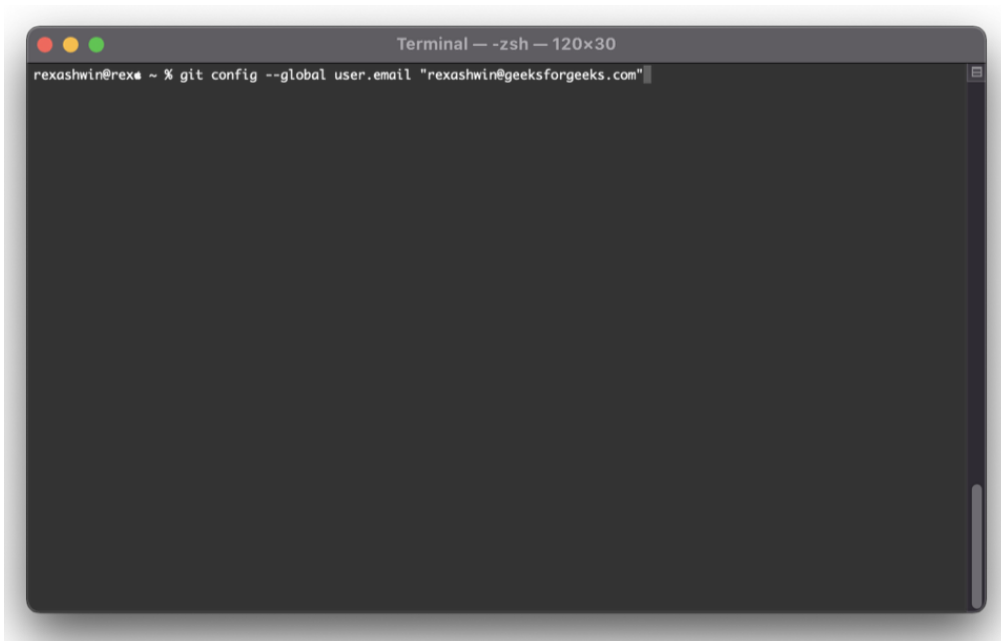
A screenshot of a macOS Terminal window. The title bar reads "Terminal — zsh — 120x30". The prompt is "rexashwin@rex ~ %". The command entered is "git config --global user.name 'RexAshwin'".

```
Terminal — zsh — 120x30
rexashwin@rex ~ % git config --global user.name "RexAshwin"
```

setting git username

Setting email id

This is for setting our email, which makes our identity as a creator of the repositories.

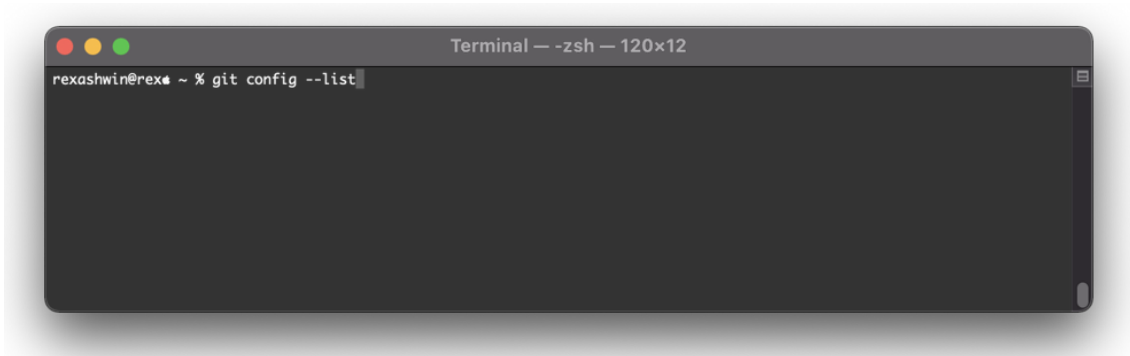
A screenshot of a macOS Terminal window. The title bar reads "Terminal — zsh — 120x30". The prompt is "rexashwin@rex ~ %". The command entered is "git config --global user.email 'rexashwin@geeksforgeeks.com'".

```
Terminal — zsh — 120x30
rexashwin@rex ~ % git config --global user.email "rexashwin@geeksforgeeks.com"
```

Setting email id

List of Git config

To verify our Git settings of the local repository, use the **git config --list** command as given below.



List of Git config

What is Git Bash?

- Git Bash is a command-line interface for Microsoft Windows.
- It is used to interact with the Git version control system.
- It allows cloning repositories.
- It helps in committing changes.
- It supports pushing and pulling code.
- It can run scripts to automate tasks.
- It helps in learning Git and version control.

Why Use Git Bash?

1. **Compatibility:** Git Bash works seamlessly with Git, making it an essential tool for version control.
2. **Powerful Command Line Tools:** It offers Unix-based command line utilities that are flexible and efficient.
3. **Ease of Use:** Git Bash provides a Linux-like Bash environment on Windows, ideal for Unix/Linux users.

Steps to Install Git Bash :-

Follow the steps given below to install **Git Bash on Windows**:

Step 1: Download and Run the installer

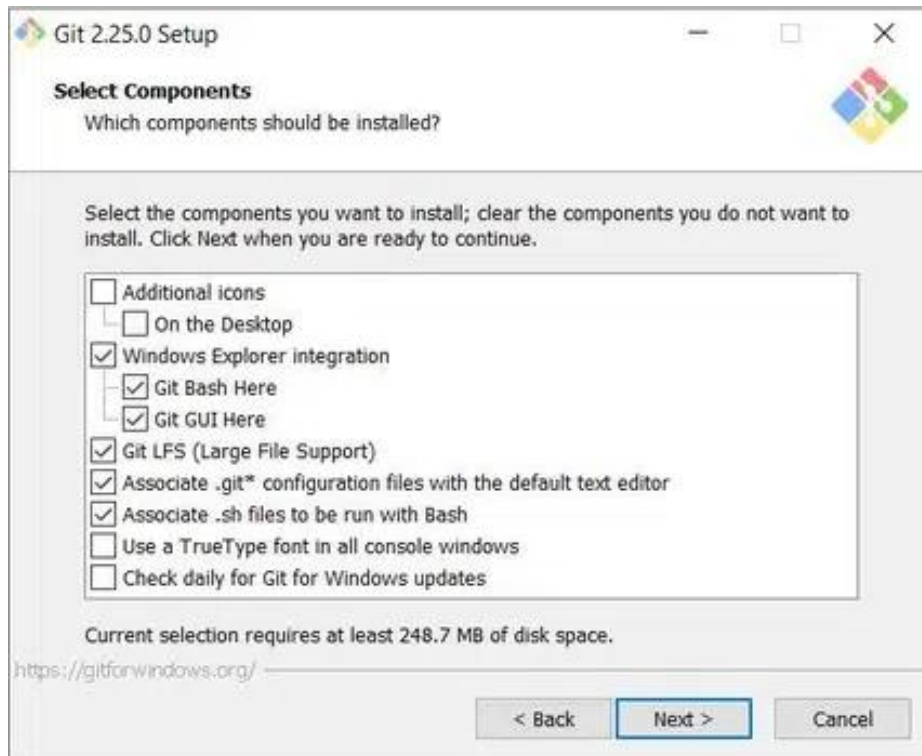
The .exe file installer for Git Bash can be downloaded from "<https://gitforwindows.org/>" Once downloaded execute that installer, following window will occur.



Git Bash Installation

Step 2: Select required options

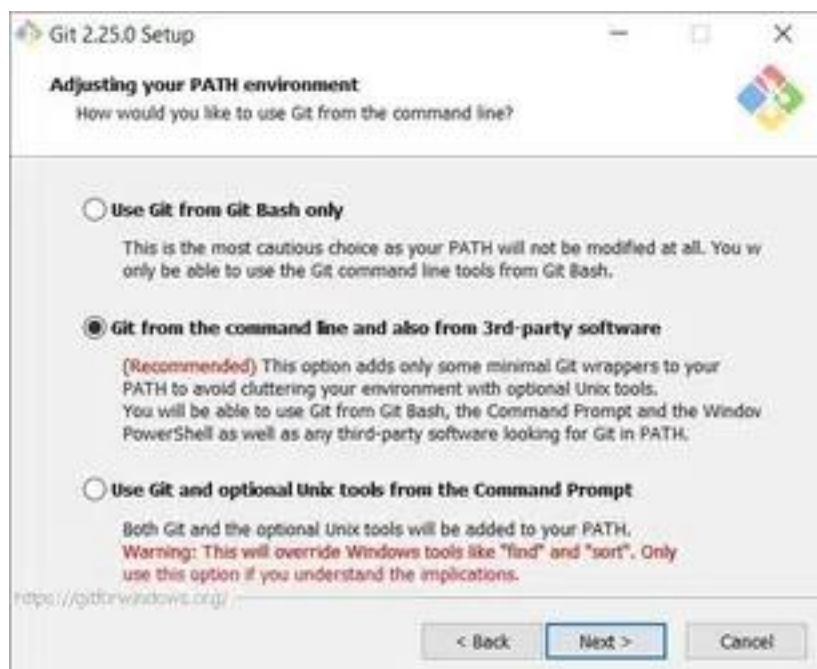
Select the components that you need to install and click on the Next button.



Install Git Bash

Step 3: Select Installation Directory

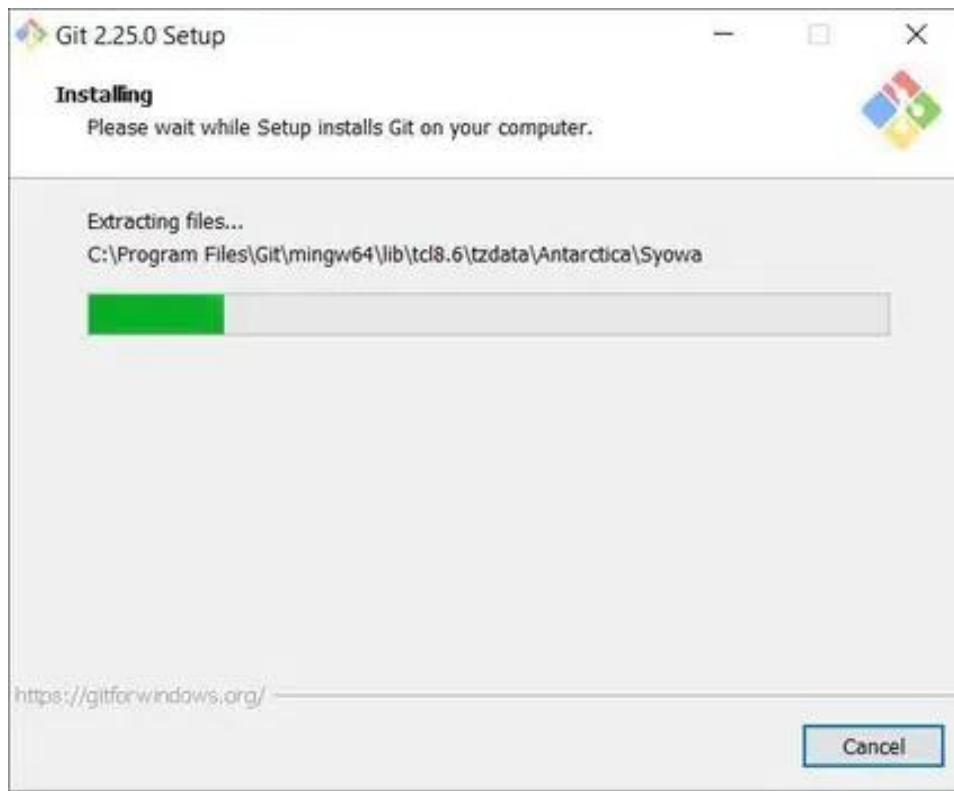
Select the path where you want to **install git** as shown in the image below.



Install Git Bash

Step 4: Wait for Completion

Let the installation process finish to begin using Git Bash. To open **Git Bash** navigate to the folder where you have installed the git otherwise just simply search in your OS for git bash.



Git Bash

App

-  Open
-  Run as administrator
-  Open file location
-  Pin to Start
-  Pin to taskbar
-  Uninstall

Install Git Bash

Basic Git Bash Commands

Here are some fundamental Git Bash commands to get you started:

1. Navigating Directories:

1. **ls**: Lists files and directories in the current directory.
2. **cd <directory>**: Changes the current directory to the specified directory.
3. **pwd**: Prints the current working directory.

2. Managing Files and Directories:

1. **touch <file>**: Creates a new file.
2. **mkdir <directory>**: Creates a new directory.
3. **rm <file>**: Removes a file.
4. **rmdir <directory>**: Removes an empty directory.

3. Using Git Commands:

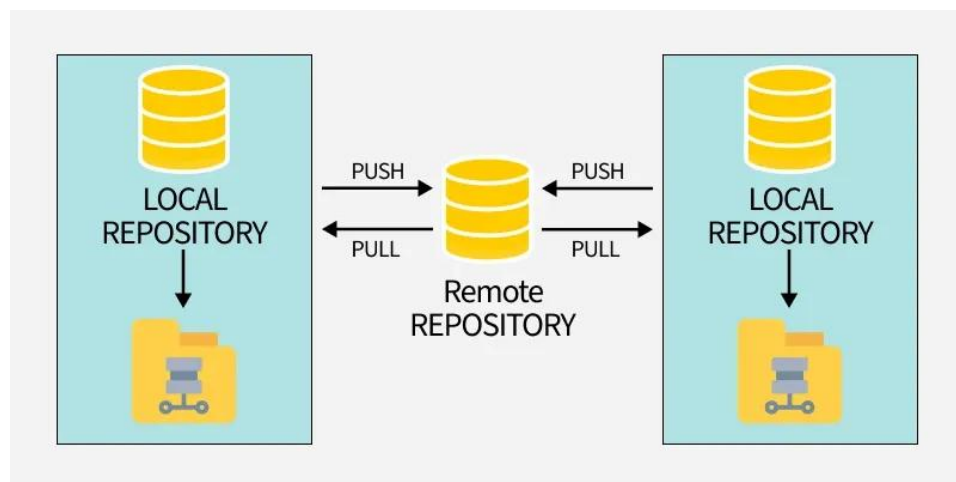
1. **git init**: Initializes a new Git repository.
2. **git clone <repository>**: Clones an existing repository.
3. **git status**: Displays the status of your working directory and staging area.
4. **git add <file>**: Adds a file to the staging area.
5. **git commit -m "message"**: Commits the changes in the staging area with a message.
6. **git push**: Pushes changes to a remote repository.
7. **git pull**: Pulls changes from a remote repository.
8. **git log**: "[git log](#)" command will help you to see the logs of all the commits made.
9. **git branch**: Creates a new branch.
10. **git merge**: For merging the changes from one branch to another branch.
11. **git config**: "[git config](#)" will help you to configure the username and email id.
12. **git tag**: It will display the all tags.

Core Concepts of Git :-

1. Repositories

A repository (repo) is a place where project files and their history are stored.

1. **Local Repository:** Stored on your local computer.
2. **Remote Repository:** Stored on a server (e.g., GitHub, GitLab, Bitbucket).



2. Commits

- A commit is a snapshot of the project at a specific time.
- Each commit has a unique hash (ID).
- It includes a message describing the changes.
- Commits help track project history.
- They allow you to review past changes.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file.txt

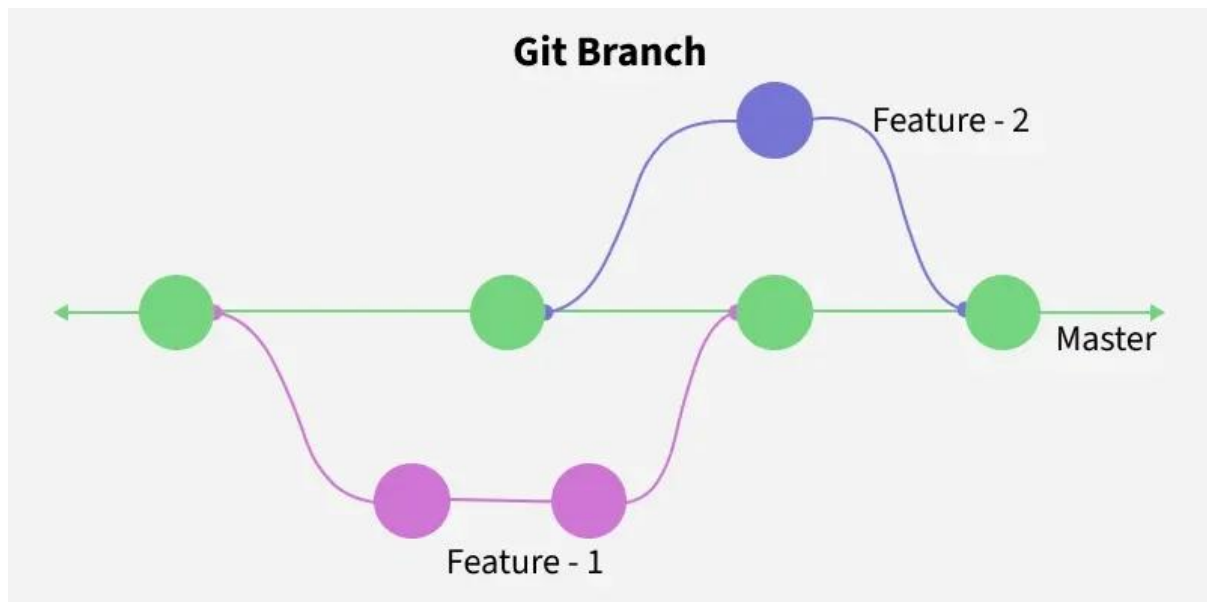
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git commit -m "first commit"
[master (root-commit) 5a457ea] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file.txt

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

3. Branches

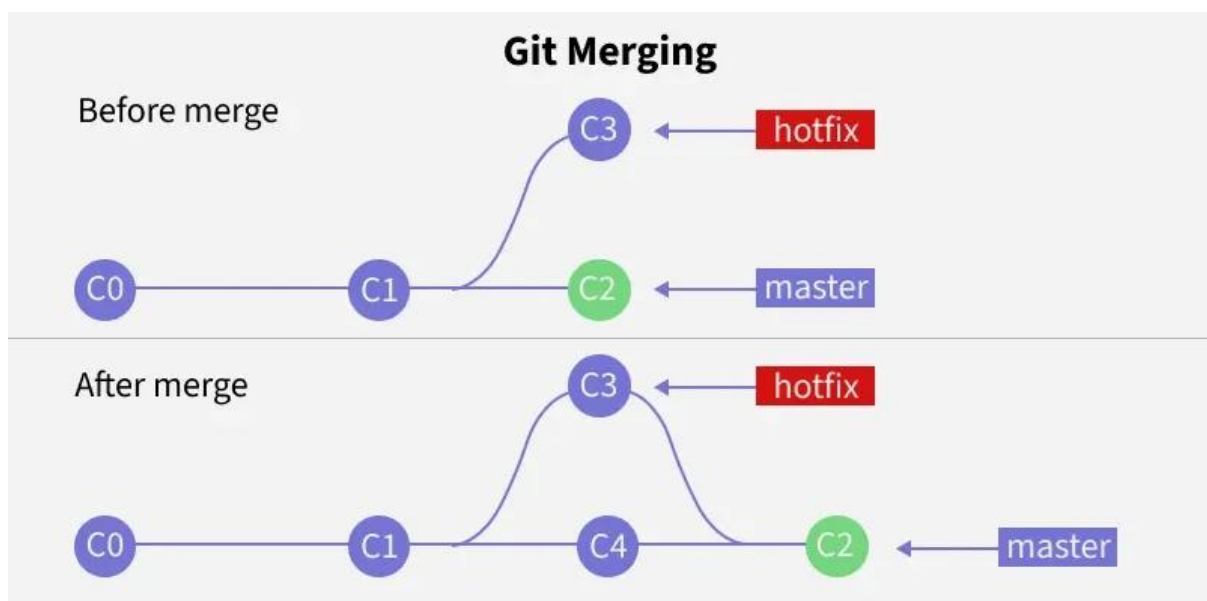
Branches allow developers to work on separate tasks without affecting the main codebase. Common branch types include:

1. **Main (or Master) Branch:** The stable version of the project, usually production-ready.
2. **Feature Branch:** Used for developing new features or bug fixes.



4. Merging

- Merging combines changes from one branch into another.
- It helps integrate work from different branches.
- Merging may cause conflicts.
- Conflicts must be resolved to complete the merge.



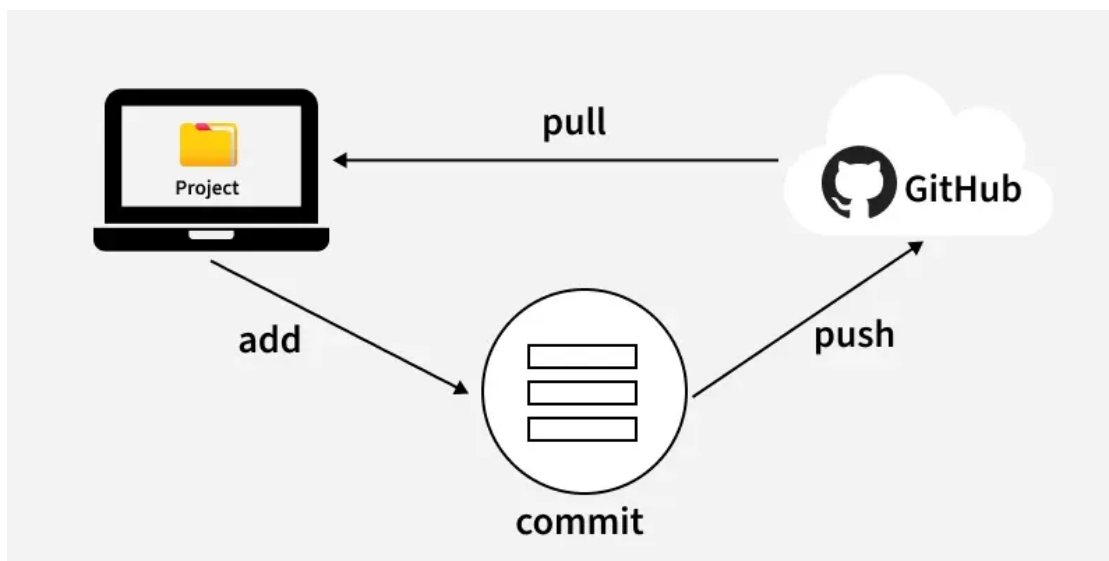
5. Cloning

- Cloning creates a local copy of a remote repository.
- It includes all project files.
- It also copies branches.
- The entire commit history is included.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git clone https://github.com/twbs/bootstrap.git
Cloning into 'bootstrap'...
remote: Enumerating objects: 184546, done.
fatal: fetch-pack: invalid index-pack output5 MiB | 2.58 MiB/s
fetch-pack: unexpected disconnect while reading sideband packet
```

6. Pull and Push

1. **Pull:** Fetches updates from the remote repository and integrates them into your local repository.
2. **Push:** Sends your local changes to the remote repository, making them available to others.



The Three States (The Staging Area) :-

In Git, files exist in three main states:

1. **Working Directory:** Files you are currently editing; changes are not yet tracked.
2. **Staging Area (Index):** Holds changes selected for the next commit using `git add`.
3. **Repository (.git):** Stores permanent snapshots (commits) using `git commit`.

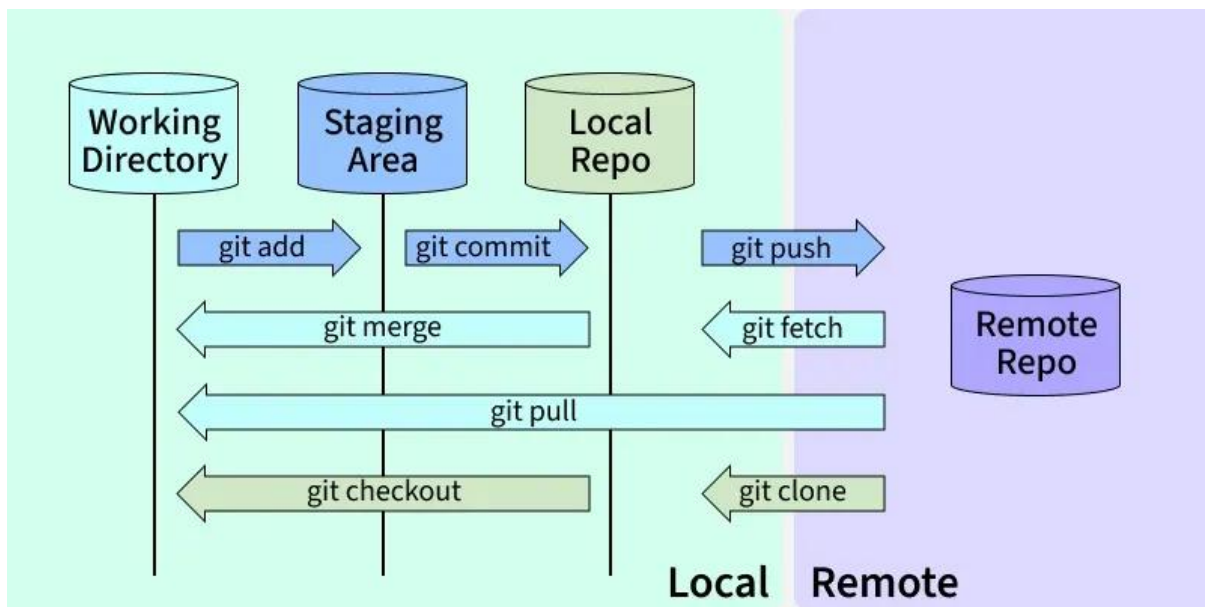
This three-step process (modify -> add -> commit) gives you precise control over what gets saved in your project's history.

Basic Git Commands :-

Git commands help manage repositories, track changes, and collaborate efficiently.

Command	Description
git status	Shows the current status of the repository — staged, unstaged, and untracked files.
git add <file-name>	Stages a specific file for commit. Use git add . to stage all changes.
git commit -m "message"	Commits the staged changes with a descriptive commit message.
git branch <branch-name>	Creates a new branch with the given name.
git checkout <branch-name>	Switches to the specified branch.
git merge <branch-name>	Merges changes from the given branch into the current branch.
git push origin <branch-name>	Pushes the local branch changes to the remote repository.
git pull origin <branch-name>	Fetches and merges changes from the remote repository into the local branch.
git log	Displays the commit history for the current branch.

Git Workflow :-



Git workflows define how developers should use Git in a structured and efficient manner.

1. Clone the Repository

```
git clone git@github.com:username/repository.git
```

2. Create and Switch to a New Branch

```
git checkout -b feature-branch
```

3. Make Changes and Stage Them

```
git add <file-name>
```

4. Commit the Changes

```
git commit -m "Add new feature"
```

5. Push the Changes

```
git push origin feature-branch
```

6. Create a Pull Request: After pushing your changes, create a pull request on GitHub to merge the feature branch into the main branch.

7. Update your Local Repository

```
git checkout main  
git pull origin main
```

8. Delete Feature Branch:

```
git branch -d feature-branch  
git push origin --delete feature-branch
```